



南開大學
Nankai University

计算机学院
计算机体系结构实验报告

总线

姓名：王一诺

学号：2312331

专业：计算机科学与技术

指导老师：董前琨

2025 年 12 月 7 日

目录

1	实验要求	2
2	实验思路	2
3	类 SRAM 总线	3
3.1	代码实现	3
3.2	仿真测试	6
4	AXI 总线	6
5	总结感想	10

1 实验要求

参考《CPU 设计实战》的第 8 章，自己动手实现总线接口相关功能。要求：

1. 根据自己情况，类 SRAM 总线和 AXI 总线完成一项支持即可，建议选择 AXI 总线（需要查阅 AXI 总线文档）。
2. 源码建议不再使用原始的五级流水线 CPU（如果使用也可，需要移植到现在的实验中去），源码请使用书中给的 lab10 或者 lab11 的源码。
3. 不要求做到多完美，随机延迟或者固定延迟的功能验证效果不强制要求，但需要把自己的探索思路、尝试的方法、效果和分析内容在实验报告里面写清楚。

2 实验思路

根据实验指导书，本次实验思路是把《CPU 设计实战》第 8 章的总线接口要求在现有 gs132 核基础上落到具体 SoC 上，并分两步完成：先做类 SRAM 接口，再做 AXI 接口。

类 SRAM 总线 （lab10）

1. 按指导书的“1x2 桥 + sram_wrap + confreg”结构，先写了一个轻量包装 mycpu_top.v，把 gs132 的字节使能端口（en/wen/addr/wdata/rdata）转换成实验所需的 req/addr_ok/data_ok 协议，并用 wstrb 推导 size，直接对接书中已有的 sram_wrap 和 bridge_1x2，这样 CPU 取指、访存就能通过桥分到 inst/data RAM 和 confreg。
2. 在本次实验中把 sram_wrap.v 的宏改为 define RUN_PERF_TEST 关闭随机屏蔽，避免首拍握手丢失。
3. 在 Vivado 里把 CPU 核源码加入工程，保持 inst/data RAM 的初始化文件指向 func_lab9 生成的镜像，运行 testbench，与 golden_trace 对比 PASS。

AXI 总线 （lab11）

1. 依照指导书“CPU→AXI→交叉开关→axi_ram/confreg”的路径，先在 gs132 核定义中关闭 SRAM 窗口，让所有访存走 AXI。
2. 在 ls132r_top.v 加出完整 AXI 主机端口（ar/aw/w/r/b），宽度与 AXI 协议一致，并用新的 mycpu_top.v 把 AXI 口直接引到 SoC 顶层。
3. 按照书中地址映射，把交叉开关输出顺序改为 confreg 在 M00、axi_ram 在 M01，并同样关闭 AXI 随机屏蔽（防止首拍握手丢失，没有验证随机延迟）。
4. 用 func_lab9 的 inst_ram.mif 覆盖 axi_ram.mif 并 Regenerate IP，保证 AXI RAM 初始化与 golden_trace 一致。
5. 运行 testbench，仿真与 golden_trace 对比 PASS。

3 类 SRAM 总线

根据实验思路，这一部分的核心就是完成 mycpu_top.v 的 RTL 代码设计，将 CPU 顶层接口修改为类 SRAM 总线接口。

3.1 代码实现

mycpu_top.v 的设计，主要思路就是包装 ls132r_top，把核心的 byte-enable SRAM 口转换成 req/addr_ok/data_ok 协议，双通道（取指/数据）映射到 soc 中的 sram_wrap;wstrb→size 的 function，确保 size 符合实验接口要求。最终实现让 CPU 能通过 bridge+sram_wrap 访问 inst/data ram 与 confreg。

模块声明与接口定义

Listing 1: mycpu_top.v

```

1  module mycpu_top(
2      input          clk,
3      input          resetn,  // low active
4
5      output         inst_sram_req,
6      output         inst_sram_wr,
7      output [1 :0]  inst_sram_size,
8      output [3 :0]  inst_sram_wstrb,
9      output [31:0]  inst_sram_addr,
10     output [31:0]  inst_sram_wdata,
11     input          inst_sram_addr_ok,
12     input          inst_sram_data_ok,
13     input [31:0]   inst_sram_rdata,
14
15     output         data_sram_req,
16     output         data_sram_wr,
17     output [1 :0]  data_sram_size,
18     output [3 :0]  data_sram_wstrb,
19     output [31:0]  data_sram_addr,
20     output [31:0]  data_sram_wdata,
21     input          data_sram_addr_ok,
22     input          data_sram_data_ok,
23     input [31:0]   data_sram_rdata,
24
25     //debug interface
26     output [31:0]  debug_wb_pc,
27     output [3 :0]  debug_wb_rf_wen,
28     output [4 :0]  debug_wb_rf_wnum,
29     output [31:0]  debug_wb_rf_wdata
30 );

```

该部分定义了 mycpu_top 模块的输入输出接口，包括时钟信号和复位信号、指令通道的握手信号、数据通道的握手信号、指令和数据通道的读数据、指令和数据通道的请求信号、写使能信号、访问数据

的大小、写字节使能信号、访问地址、写数据、调试接口信号。明确了与外部模块的交互信号。

核心信号定义

Listing 2: mycpu_top.v

```

1  // core <-> wrapper signals
2  wire      core_inst_en;
3  wire [3 :0] core_inst_wen;
4  wire [31:0] core_inst_addr;
5  wire [31:0] core_inst_wdata;
6  wire [31:0] core_inst_rdata;
7
8  wire      core_data_en;
9  wire [3 :0] core_data_wen;
10 wire [31:0] core_data_addr;
11 wire [31:0] core_data_wdata;
12 wire [31:0] core_data_rdata;

```

定义了 mycpu_top 模块内部与 ls132r_top 核心模块之间的信号，这些信号用于连接 mycpu_top 模块与核心 CPU 模块 ls132r_top，实现指令和数据通道的信号传递。

核心模块实例化

Listing 3: mycpu_top.v

```

1  ls132r_top u_ls132r_core(
2      .clk          (clk          ),
3      .resetn       (resetn       ),
4
5      .inst_sram_en  (core_inst_en  ),
6      .inst_sram_wen (core_inst_wen ),
7      .inst_sram_addr (core_inst_addr ),
8      .inst_sram_wdata (core_inst_wdata ),
9      .inst_sram_rdata (core_inst_rdata ),
10
11     .data_sram_en   (core_data_en   ),
12     .data_sram_wen  (core_data_wen  ),
13     .data_sram_addr (core_data_addr ),
14     .data_sram_wdata (core_data_wdata ),
15     .data_sram_rdata (core_data_rdata ),
16
17     //debug interface
18     .debug_wb_pc     (debug_wb_pc     ),
19     .debug_wb_rf_wen (debug_wb_rf_wen ),
20     .debug_wb_rf_wnum (debug_wb_rf_wnum ),
21     .debug_wb_rf_wdata (debug_wb_rf_wdata)
22 );

```

实例化 ls132r_top 核心 CPU 模块，并将 mycpu_top 模块的时钟和复位信号连接到核心模块，同时将定义的核心信号与核心模块的对应端口连接，实现信号传递。ls132r_top 是一个黑盒模块，负责实现 CPU 的核心功能。通过信号连接，mycpu_top 模块可以将外部的 SRAM 接口信号转换为 ls132r_top 所需的信号格式。这是是整个模块的核心部分。

字节使能到大小编码的转换函数

Listing 4: mycpu_top.v

```

1  function [1:0] wstrb_to_size;
2      input [3:0] strobe;
3      begin
4          case (strobe)
5              4'b0001, 4'b0010, 4'b0100, 4'b1000: wstrb_to_size = 2'd0; // byte
6              4'b0011, 4'b0110, 4'b1100:          wstrb_to_size = 2'd1; // half
7              default:                            wstrb_to_size = 2'd2; // word/aligned
8              fetch
9          endcase
10         end
11     endfunction

```

定义了一个函数 wstrb_to_size，用于将 4 位的字节写使能信号转换为 2 位的大小编码。转换规则：

1. 单字节写使能（如 4'b0001）：大小编码为 2'd0。
2. 半字写使能（如 4'b0011）：大小编码为 2'd1。
3. 字写使能（如 4'b1111）：大小编码为 2'd2。

该函数用于适配 ls132r_top 的字节使能信号到 SRAM 接口所需的大小编码。

指令通道信号映射

Listing 5: mycpu_top.v

```

1  assign inst_sram_req  = core_inst_en;
2  assign inst_sram_wr   = |core_inst_wen;
3  assign inst_sram_wstrb = core_inst_wen;
4  assign inst_sram_size = wstrb_to_size(core_inst_wen);
5  assign inst_sram_addr = core_inst_addr;
6  assign inst_sram_wdata = core_inst_wdata;
7  assign core_inst_rdata = inst_sram_rdata;

```

将核心模块的指令通道信号映射到外部的 SRAM 接口信号。其中，inst_sram_wr 通过 |core_inst_wen 判断是否有写操作，inst_sram_size 通过调用 wstrb_to_size 函数转换字节使能为大小编码，其他信号直接映射。

数据通道信号映射

Listing 6: mycpu_top.v

```

1  assign data_sram_req    = core_data_en;
2  assign data_sram_wr     = |core_data_wen;
3  assign data_sram_wstrb  = core_data_wen;
4  assign data_sram_size   = wstrb_to_size(core_data_wen);
5  assign data_sram_addr   = core_data_addr;
6  assign data_sram_wdata  = core_data_wdata;
7  assign core_data_rdata  = data_sram_rdata;

```

与指令通道类似，将核心模块的数据通道信号映射到外部的 SRAM 接口信号。

握手信号处理

地址和数据握手信号在核心模块中始终被接受，因此无需额外的信号控制逻辑。

3.2 仿真测试

完成设计后运行 tb 仿真测试，与 golden_trace 对比，结果如下：

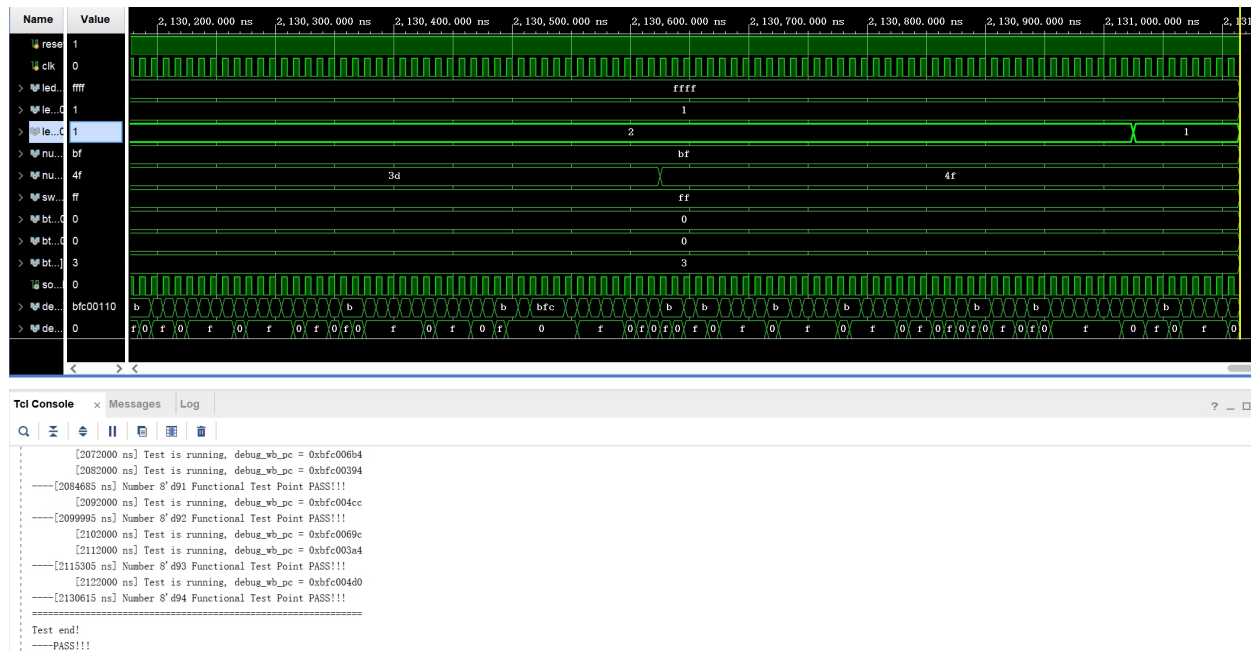


图 3.1: 测试结果

4 AXI 总线

根据实验思路，这一部分的核心是修改 `ls132r_top.v` 关闭 SRAM 窗口改为绑定 AXI 接口、完成 `mycpu_top.v` 设计包装顶层、修改 `soc_axi_lite_top.v` 修正交叉开关端口拼接。

关闭 SRAM 窗口、强制走 AXI

Listing 7: `ls132r_define.h`

```

1  // For AXI 接口实验，默认将访存请求发往 AXI 总线（仅当地址为 0 时才会走到片上
    SRAM）

```

```

2      `define LS132R_INST_SRAM_MASK      32'hffff_ffff
3      `define LS132R_INST_SRAM_WINDOW    32'h0000_0000
4
5      `define LS132R_DATA_SRAM_MASK      32'hffff_ffff
6      `define LS132R_DATA_SRAM_WINDOW    32'h0000_0000

```

原工程默认允许命中片上 SRAM 窗口。实验要求 AXI 接口，所以把掩码设成全 1，窗口 0，这样地址判定必不命中 SRAM，所有取指/访存都会走 AXI 接口，不会绕回本地 SRAM。

ls132r_top 导出 AXI 主机接口并绑定。

Listing 8: ls132r_top.v

```

1      //axi master interface
2      output [3 :0]   arid,
3      output [31:0]   araddr,
4      output [7 :0]   arlen,
5      output [2 :0]   arsize,
6      output [1 :0]   arburst,
7      output [1 :0]   arlock,
8      output [3 :0]   arcache,
9      output [2 :0]   arprot,
10     output          arvalid,
11     input           arready,
12     ...
13     output [3 :0]   wid,
14     output [31:0]   wdata,
15     output [3 :0]   wstrb,
16     output          wlast,
17     output          wvalid,
18     input           wready,
19     ...
20     input  [3 :0]   bid,
21     input  [1 :0]   bresp,
22     input          bvalid,
23     output         bready,
24
25     ...
26
27     wire [3:0]   arid_int;    ... // 内部 AXI 信号
28     ...
29     assign arid      = arid_int;
30     assign araddr    = araddr_int;
31     assign arlen     = {4'd0, arlen_int}; // 内部 4 位 len 扩展到 AXI 8 位
32     ...
33     assign rid_int   = rid;
34     assign rdata_int = rdata;
35     ...
36     assign awid      = awid_int;

```



```

37     assign awaddr      = awaddr_int;
38     assign awlen       = {4'd0, awlen_int};
39     ...
40     assign bid_int     = bid;
41     assign bresp_int   = bresp;
42     assign bvalid_int  = bvalid;
43     assign bready      = bready_int;

```

ls132r 核内部接口宽度与自身控制逻辑相关（如 len 4 位），对外需要符合 AXI 协议（len 8 位），这里做了信号导出和必要的宽度扩展。通过 *_int 中间信号把内部逻辑与外部端口解耦，便于封装到 SoC。

AXI 版本 mycpu_top .

Listing 9: mycpu_top.v

```

1  module mycpu_top(
2      input  [5 :0] int,    // 未使用中断
3      input          aclk,
4      input          aresetn,
5      output [3 :0] arid, ... output bready,
6      //debug interface
7      output [31:0] debug_wb_pc,
8      ...
9  );
10 wire [31:0] fake_inst_rdata = 32'b0;
11 wire [31:0] fake_data_rdata = 32'b0;
12
13 ls132r_top u_ls132r_axi(
14     .clk      (aclk), .resetn (aresetn),
15     // 屏蔽未用的 SRAM 端口
16     .inst_sram_en    ( ),
17     .inst_sram_wen    ( ),
18     .inst_sram_addr   ( ),
19     .inst_sram_wdata   ( ),
20     .inst_sram_rdata  (fake_inst_rdata),
21     .data_sram_en     ( ),
22     .data_sram_wen    ( ),
23     .data_sram_addr   ( ),
24     .data_sram_wdata   ( ),
25     .data_sram_rdata  (fake_data_rdata),
26     // 直接引出 AXI
27     .arid   (arid),   .araddr (araddr), .arlen  (arlen), ...
28     .awid   (awid),   .awaddr (awaddr), .awlen  (awlen), ...
29     .wid    (wid),    .wdata  (wdata), .wstrb  (wstrb),
30     .wlast  (wlast),  .wvalid (wvalid), .wready (wready),
31     .bid    (bid),    .bresp  (bresp), .bvalid (bvalid), .bready (bready),
32     // debug 透传
33     .debug_wb_pc      (debug_wb_pc),
34     .debug_wb_rf_wen  (debug_wb_rf_wen),

```

```

35     .debug_wb_rf_wnum (debug_wb_rf_wnum),
36     .debug_wb_rf_wdata(debug_wb_rf_wdata)
37 );
38 endmodule

```

AXI 实验不再使用类 SRAM 接口，所以把 SRAM 端口悬空（读数据用零占位），只保留 AXI 通道。这样 SoC 顶层直接实例化该包装器，无需再适配。

交叉开关端口拼接修正

Listing 10: soc_axi_lite_top.v

```

1  axi_crossbar_1x2 u_axi_crossbar_1x2(
2      ...
3      // crossbar IP 配置的 M00 对应 confreg, M01 对应 axi_ram, 这里按配置顺序拼接
4      .m_axi_arid      ( {conf_arid  ,ram_arid  } ),
5      .m_axi_araddr    ( {conf_araddr ,ram_araddr } ),
6      .m_axi_arlen     ( {conf_arlen  ,ram_arlen  } ),
7      .m_axi_arsize    ( {conf_arsize ,ram_arsize } ),
8      .m_axi_arburst   ( {conf_arburst,ram_arburst} ),
9      .m_axi_arlock    ( {conf_arlock ,ram_arlock } ),
10     .m_axi_arcache    ( {conf_arcache,ram_arcache} ),
11     .m_axi_arprot     ( {conf_arprot ,ram_arprot } ),
12     .m_axi_arvalid    ( {conf_arvalid,ram_arvalid} ),
13     .m_axi_arready    ( {conf_arready,ram_arready} ),
14     .m_axi_rid        ( {conf_rid    ,ram_rid    } ),
15     .m_axi_rdata      ( {conf_rdata  ,ram_rdata  } ),
16     .m_axi_rresp      ( {conf_rresp  ,ram_rresp  } ),
17     .m_axi_rlast      ( {conf_rlast  ,ram_rlast  } ),
18     .m_axi_rvalid     ( {conf_rvalid ,ram_rvalid } ),
19     .m_axi_rready     ( {conf_rready ,ram_rready } ),
20     .m_axi_awid       ( {conf_awid   ,ram_awid   } ),
21     .m_axi_awaddr     ( {conf_awaddr ,ram_awaddr } ),
22     .m_axi_awlen      ( {conf_awlen  ,ram_awlen  } ),
23     .m_axi_awsiz      ( {conf_awsiz  ,ram_awsiz  } ),
24     .m_axi_awburst    ( {conf_awburst,ram_awburst} ),
25     .m_axi_awlock     ( {conf_awlock ,ram_awlock } ),
26     .m_axi_awcache    ( {conf_awcache,ram_awcache} ),
27     .m_axi_awprot     ( {conf_awprot ,ram_awprot } ),
28     .m_axi_awvalid    ( {conf_awvalid,ram_awvalid} ),
29     .m_axi_awready    ( {conf_awready,ram_awready} ),
30     .m_axi_wid        ( {conf_wid    ,ram_wid    } ),
31     .m_axi_wdata      ( {conf_wdata  ,ram_wdata  } ),
32     .m_axi_wstrb      ( {conf_wstrb  ,ram_wstrb  } ),
33     .m_axi_wlast      ( {conf_wlast  ,ram_wlast  } ),
34     .m_axi_wvalid     ( {conf_wvalid ,ram_wvalid } ),
35     .m_axi_wready     ( {conf_wready ,ram_wready } ),
36     .m_axi_bid        ( {conf_bid    ,ram_bid    } ),
37     .m_axi_bresp      ( {conf_bresp  ,ram_bresp  } ),

```

```
38     .m_axi_bvalid      ( {conf_bvalid ,ram_bvalid } ),  
39     .m_axi_bready     ( {conf_bready ,ram_bready } )  
40 );
```

交叉开关 IP 配置中 M00 端口是 confreg(基地址 0x1faf0000/0xbfaf0000),M01 是 axi_ram(0x1fc00000/0x80000000 等)。原拼接顺序反了,导致 CPU 访问 0xbfc...被路由到 confreg。调整拼接顺序与 IP 配置一致,路由正确,取指/数据才能命中 axi_ram。

这些修改共同完成了 AXI 总线实践:CPU 端完全 AXI 化、关闭 SRAM 窗口、交叉开关路由正确、握手确定性,并用 func_lab9 镜像初始化 axi_ram,最终 testbench 与 golden_trace 对比 PASS,此处不再赘述。

5 总结感想

通过本次实验,让我更具体的学习和理解了总线的设计。

实验的最大收获是把纸面协议和实际工程“打通”的感觉。读实验指导书时,AXI 的信号名、握手机制、地址映射看起来都很规范,但真正落到 SoC 上,必须去对齐每一层:CPU 的地址窗口、桥/交叉开关的端口顺序、RAM/外设的基址和延迟策略。一次错配就能让指令跑偏,强调了“明确地址空间划分”“确认握手时序”和“确保存储镜像一致”这三点。

在进行 AXI 设计时出现了路由问题,crossbar 拼接顺序和地址表不一致,取指走错了路,修正路由后立刻对上 golden_trace。说明在多端口、多层级互连时,映射表和拼接顺序是容易踩坑的。

另一点是,保持初始化数据与参考一致同样关键。无论是 SRAM 的 inst/data_ram,还是 AXI 的 axi_ram,镜像不一致会让一切验证失效,哪怕接口、握手都完美。

最后,包装层的设计也体现了最小侵入,用一个 mycpu_top 把核心接口适配到 SoC 现有的桥/交叉开关,不改 CPU 内核,只动边缘,降低引入新 bug 的概率。